11. 보안

AI 에이전트에게 터미널 접근 권한을 주는 것은 강력하지만, 그만큼 위험할 수 있습니다. 잘못된 명령 하나가 파일을 삭제하거나, API 키를 노출하거나, 시스템을 손상시킬 수 있습니다. 특히 메시징 게이트웨이를 통해 외부에서 접근하거나, 크론으로 무인 실행하는 경우에는 보안이 더욱 중요합니다. 이 장에서는 Hermes Agent의 보안 모델을 다룹니다. Hermes Agent는 심층 방어(defense-in-depth) 원칙에 따라 5개 계층의 보안 장치를 제공합니다.

이 장에서 다루는 내용

- 5계층 보안 모델
- 컨테이너 격리
- 운영 환경 보안 체크리스트

11-1. 5계층 보안 모델

> ⚠️ ***부분 검증**: v0.16.0 소스 코드 분석으로 확인했습니다. 보안 정책 상수·패턴 표·기본값·허용 목록을 파일 단위로 대조한 결과이며, 별도 실행 테스트는 하지 않았습니다(이 절의 값은 대부분 정적 상수라 실행보다 코드 대조로 검증하는 편이 정확합니다). 본문의 위험 명령 패턴·시크릿 접두사·차단 경로 목록은 소스의 전체 목록 중 핵심 항목을 추린 것입니다.*

- 1계층: 사용자 인증
  - DM 페어링 시스템
- 2계층: 위험 명령 승인
  - 차단 대상 패턴 (전체 목록)
  - 승인 모드 설정
  - CLI 승인 흐름
  - 게이트웨이 승인 흐름
  - 영구 허용 목록
  - Tirith 보안 스캔
- 3계층: 컨테이너 격리
- 4계층: MCP 자격증명 필터링
  - 안전한 환경변수 (허용 목록)
  - 에러 메시지 자격증명 제거
  - 시크릿 자동 마스킹 (redact.py)
  - 자격증명 파일 읽기 차단
  - 컨트롤 플레인 파일 쓰기 차단
  - 커스텀 엔드포인트 API 키 격리
  - Bitwarden Secrets Manager 연동
- 5계층: 컨텍스트 파일 스캔
  - 탐지 패턴 (소스 코드 확인)
  - 비가시 유니코드 탐지

Hermes Agent는 심층 방어(defense-in-depth) 원칙에 따라 다층 보안 모델을 구현합니다. v0.16.0 기준 공식 문서는 7개 계층을 명시합니다. 이 절에서는 핵심이 되는 5개 계층(인증 → 승인 → 컨테이너 → 자격증명 필터 → 컨텍스트 스캔)을 다루고, 뒤에 추가된 두 계층(세션 간 격리, 입력 살균)은 이름만 언급합니다.

계층	역할	핵심 파일
1계층	사용자 인증	`gateway/pairing.py`
2계층	위험 명령 승인	`tools/approval.py`
3계층	컨테이너 격리	`tools/environments/docker.py`
4계층	MCP 자격증명 필터링	`tools/mcp_tool.py`
5계층	컨텍스트 파일 스캔	`agent/prompt_builder.py`

여기에 추가로 **시크릿 자동 마스킹**(`agent/redact.py`)과 **Tirith 보안 스캔**(`tools/tirith_security.py`)이 2계층 승인 시스템에 통합되어 작동합니다.

1계층: 사용자 인증

누가 에이전트와 대화할 수 있는지를 통제합니다. 인증 확인 순서는 다음과 같습니다.

1. 플랫폼별 전체 허용 플래그 (예: `DISCORD_ALLOW_ALL_USERS=true`)
2. DM 페어링 승인 목록
3. 플랫폼별 허용 목록 (예: `TELEGRAM_ALLOWED_USERS=12345,67890`)
4. 글로벌 허용 목록 (`GATEWAY_ALLOWED_USERS=12345,67890`)
5. 글로벌 전체 허용 (`GATEWAY_ALLOW_ALL_USERS=true`)
6. **기본값: 거부**

```
# ~/.hermes/.env에서 설정
TELEGRAM_ALLOWED_USERS=123456789,987654321
DISCORD_ALLOWED_USERS=111222333444555666

# 크로스 플랫폼 허용 목록
GATEWAY_ALLOWED_USERS=123456789
```

허용 목록 미설정 시 기본 작동

허용 목록이 하나도 설정되어 있지 않고 `GATEWAY_ALLOW_ALL_USERS` 도 설정하지 않으면, 모든 사용자가 거부됩니다. 게이트웨이 시작 시 경고 로그가 출력됩니다.

> ⚠️ ***Discord 역할 우회 제거**: 이전에는 `DISCORD_ALLOWED_ROLES` 만 설정하면 역할 멤버십으로 `allowed_users` 검사를 건너뛸 수 있었으나, 이 경로가 보안 패치로 제거됐습니다. Discord에서도 사용자 ID 기반의 허용 목록 또는 DM 페어링을 거쳐야 합니다.*

DM 페어링 시스템

사용자 ID를 미리 알 필요 없이, 코드 기반으로 인증할 수 있습니다.

1. 모르는 사용자가 DM을 보내면, 봇이 8자리 페어링 코드를 응답합니다
2. 봇 관리자가 CLI에서 `hermes pairing approve <platform> <code>` 를 실행합니다
3. 해당 사용자가 영구적으로 승인됩니다

보안 사양 (OWASP + NIST SP 800-63-4 기반):

- **코드 형식**: 32문자 알파벳에서 8자리 생성 (0/O, 1/I 제외하여 혼동 방지)
- **난수**: `secrets.choice()` 사용 (암호학적 안전)
- **코드 저장**: 솔트 + SHA-256 해시로 저장. 평문은 보관하지 않으며, 승인 시 상수 시간 비교 (`secrets.compare_digest`)로 검증합니다. 이전 버전의 평문 항목은 업그레이드 시 자동 마이그레이션됩니다
- **코드 만료**: 1시간
- **속도 제한**: 사용자당 10분에 1회 요청
- **최대 대기**: 플랫폼당 3개 대기 코드
- **잠금**: 승인 실패 5회 시 1시간 잠금
- **파일 보안**: 모든 데이터 파일에 `chmod 0600` 적용

```
# ~/.hermes/config.yaml에서 작동 설정
unauthorized_dm_behavior: pair    # pair | ignore

whatsapp:
  unauthorized_dm_behavior: ignore  # 플랫폼별 오버라이드 가능
```

2계층: 위험 명령 승인

파괴적인 명령을 실행하기 전에 사용자의 승인을 요구합니다. `tools/approval.py` 에 정의된 패턴 매칭으로 작동합니다.

차단 대상 패턴 (전체 목록)

위험 명령은 두 단계로 다룹니다.

- **DANGEROUS_PATTERNS** — 패턴이 매칭되면 사용자에게 승인을 요청하는 일반 위험 명령

- **HARDLINE_PATTERNS** — v0.11에서 추가된 *복구 불가능한* 명령 블록리스트. 이 목록에 잡힌 명령은 승인 모드와 무관하게 차단되며, 영구 허용 목록에도 추가할 수 없습니다(`/yolo`로 우회 불가)

성능을 위해 두 패턴 그룹은 모두 시작 시 정규식으로 사전 컴파일됩니다. 아래 목록은 DANGEROUS_PATTERNS의 주요 항목입니다(HARDLINE_PATTERNS는 그중에서도 절대 허용해서는 안 되는 일부를 추려 별도 차단합니다).

패턴	설명
`rm -r`, `rm --recursive`	재귀적 삭제
`rm ... /`	루트 경로 삭제
`chmod 777` (재귀 포함)	전체 쓰기 권한
`chown -R root`	루트로 재귀 소유권 변경
`mkfs`	파일시스템 포맷
`dd if=`	디스크 복사
`> /dev/sd*`	블록 디바이스 쓰기
`DROP TABLE/DATABASE`	SQL DROP
`DELETE FROM` (WHERE 없이)	SQL DELETE
`TRUNCATE TABLE`	SQL TRUNCATE
`> /etc/`	시스템 설정 덮어쓰기
`systemctl stop/disable/mask`	시스템 서비스 중지
`kill -9 -1`, `pkill -9`	프로세스 전체 킬
포크 폭탄 패턴	포크 폭탄
`bash -c`, `sh -lc` 등	셸 명령 실행 (-c 플래그)
`python -e`, `perl -e` 등	스크립트 실행 (-e/-c 플래그)
`curl | sh`, `wget | bash`	원격 코드 파이프 실행
`bash <(curl ...)`	프로세스 치환을 통한 원격 실행
`tee /etc/...` 등	tee로 시스템 파일 덮어쓰기
`xargs rm`	xargs + rm 조합

패턴	설명
`find -exec rm` , `find -delete`	find + 삭제 조합

승인 모드 설정

설정 키는 `approvals.mode` 이고 기본값은 `"manual"` 입니다(값: `manual | smart | off`). 공식 문서도 같은 키·기본값을 사용합니다.

```
# ~/.hermes/config.yaml
approvals:
  mode: manual    # manual | smart | off
```

모드	작동
`manual` (기본값)	위험 명령 감지 시 사용자에게 승인 요청
`smart`	보조 LLM이 위험도를 평가하여 안전하면 자동 승인, 위험하면 차단, 불확실하면 사용자에게 에스컬레이션
`off`	모든 승인 검사 생략 (`HERMES_YOLO_MODE` 와 동일)

smart 모드의 작동 원리

smart 모드에서는 보조 LLM(`auxiliary_client`)이 명령을 평가합니다. `python -c "print('hello')"` 처럼 패턴은 매칭되지만 실제로는 안전한 명령을 자동으로 승인합니다. 판단 결과는 APPROVE(세션 승인), DENY(차단), ESCALATE(사용자 확인) 세 가지입니다.

CLI 승인 흐름

```
⚠️  DANGEROUS COMMAND: recursive delete
    rm -rf /tmp/old-project

    [o]nce  |  [s]ession  |  [a]lways  |  [d]eny

    Choice [o/s/a/D]:
```

- **once**: 이번 한 번만 허용
- **session**: 이 세션 동안 같은 패턴 허용
- **always**: 영구 허용 목록에 추가 (`config.yaml`에 저장)
- **deny** (기본값): 차단 (60초 타임아웃 시 자동 거부)

게이트웨이 승인 흐름

메시징 플랫폼에서는 위험 명령 정보가 채팅으로 전송되고, 사용자의 응답을 기다립니다.

영구 허용 목록

`always`로 승인한 패턴은 `config.yaml`에 저장됩니다.

```
command_allowlist:
  - "recursive delete"
  - "shell command via -c/-lc flag"
```

Tirith 보안 스캔

위험 명령 패턴 매칭과 별도로, **Tirith**라는 외부 바이너리가 콘텐트 수준의 보안 위협을 스캔합니다. 호모그래프 URL, 터미널 인젝션, 파이프-투-인터프리터 등을 탐지합니다.

- 종료 코드로 판정: 0=허용, 1=차단, 2=경고
- Tirith가 설치되어 있지 않으면 GitHub에서 자동 다운로드 (SHA-256 + 선택적 cosign 검증)
- 기본 설정은 fail-open (Tirith 실패 시 명령 허용)

```
# ~/.hermes/config.yaml
security:
  tirith_enabled: true       # 기본값: true
  tirith_timeout: 5          # 스캔 타임아웃 (초)
  tirith_fail_open: true     # 실패 시 허용 (기본값: true)
```

Tirith를 쓸 수 없는 경우 — 아직 설치되지 않았거나 백그라운드 설치가 끝나기 전, 또는 지원하지 않는 플랫폼 (macOS·Linux만 지원하며 Windows는 빌드가 없습니다) — 에는 콘텐트 스캔이 생략되고 명령 검사가 위쪽의 위험 명령 패턴 매칭만으로 동작합니다. 이때 CLI 시작 시 `tirith security scanner enabled but not available — command scanning will use pattern matching only` 경고가 표시됩니다.

> 컨테이너 내에서는 승인 불필요
>
> Docker, Singularity, Modal, Daytona 백엔드에서는 위험 명령 승인과 Tirith 스캔이 모두 생략됩니다. 컨테이너 자

체가 격리 경계이기 때문입니다. `HERMES_YOLO_MODE` 환경변수나 `approvals.mode: off` 설정도 동일한 효과입니다.

3계층: 컨테이너 격리

Docker, Singularity, Modal, Daytona 백엔드를 사용하면 호스트로부터 격리된 환경에서 명령이 실행됩니다. 상세 내용은 11-2절에서 다룹니다.

4계층: MCP 자격증명 필터링

MCP 서버 서브프로세스에는 **필터링된 환경변수**만 전달됩니다.

안전한 환경변수 (허용 목록)

```
PATH, HOME, USER, LANG, LC_ALL, TERM, SHELL, TMPDIR
```

그리고 `XDG_*` 접두사 변수만 전달됩니다. `OPENAI_API_KEY`나 `ANTHROPIC_API_KEY` 같은 변수는 명시적으로 설정하지 않으면 서브프로세스에 노출되지 않습니다.

```
mcp_servers:
  github:
    command: "npx"
    args: ["-y", "@modelcontextprotocol/server-github"]
    env:
      GITHUB_PERSONAL_ACCESS_TOKEN: "ghp_..."  # 이것만 전달됨
```

에러 메시지 자격증명 제거

MCP 도구의 에러 메시지에서 다음 패턴이 `[REDACTED]`로 치환됩니다.

- GitHub PAT (`ghp_...`)
- OpenAI 스타일 키 (`sk-...`)
- Bearer 토큰
- `token=` , `key=` , `API_KEY=` , `password=` , `secret=` 파라미터

시크릿 자동 마스킹 (redact.py)

MCP 외에도 터미널 출력, 파일 읽기, 로그 전반에서 `agent/redact.py`가 시크릿을 마스킹할 수 있습니다. 대상은 다음과 같습니다.

- 30개 이상의 알려진 API 키 접두사 (sk-, ghp_, github_pat_, xox[baprs]-, AIza, hf_, AKIA 등)
- 환경변수 할당 (`OPENAI_API_KEY=sk-abc...`)
- JSON 필드 (`"apiKey": "value"`)
- Authorization 헤더
- Telegram 봇 토큰
- RSA 프라이빗 키 블록
- 데이터베이스 연결 문자열의 패스워드
- E.164 전화번호 (Signal, WhatsApp)

18자 미만 토큰은 완전히 마스킹(`***`)되고, 긴 토큰은 앞 6자 + 뒤 4자만 남깁니다.

> ⚠️ ***시크릿 마스킹 기본값(ON)**: v0.16.0 기준 시크릿 마스킹의 **기본값은 ON**입니다(안전한 기본값, 이슈 #17691). 한때 정규식이 가짜 시크릿 모양 부분 문자열을 잘못 잡아 도구 출력이 망가지는 문제로 기본값을 OFF 로 둔 적이 있으나, 이후 다시 ON으로 되돌렸습니다. 끄려면 `config.yaml` 에서 `security.redact_secrets: false` 로 두거나(2-4절 참고), `HERMES_REDACT_SECRETS=false` 환경 변수를 설정합니다. 안전 경계에 해당하는 출력(상태 배너 등)은 이 설정과 무관하게 강제 마스킹됩니다.*

자격증명 파일 읽기 차단

에이전트가 `read_file` 도구로 Hermes 자신의 자격증명 파일에 접근하려 하면 도구 호출 자체를 거부합니다. 차단 대상은 다음과 같습니다.

경로	내용
`~/.hermes/auth.json`	OAuth 토큰·API 키
`~/.hermes/auth.lock`	인증 잠금 파일
`~/.hermes/.anthropic_oauth.json`	Anthropic OAuth 토큰
`~/.hermes/.env`	환경변수·API 키
`~/.hermes/webhook_subscriptions.json`	웹훅 시크릿
`~/.hermes/mcp-tokens/`	MCP OAuth 토큰 디렉터리

프로필 모드(`~/.hermes/profiles/<이름>/`)에서 실행 중이더라도, 프로필 경로와 글로벌 루트(`~/.hermes/`) 양쪽의 자격증명 파일이 모두 차단됩니다. 프로필 세션이 글로벌 `.env` 나 `auth.json` 을 읽어 다른 프로필의 키에 접근하는 경로

를 막기 위함입니다.

또한 `TERMINAL_CWD` 가 `~/.hermes` 인 상태에서 상대 경로(`auth.json`)로 읽기를 시도하는 경우도 차단됩니다. 내부적으로 경로를 먼저 절대 경로로 변환한 뒤 거부 목록과 대조합니다.

> 💡 ***심층 방어**: 이 차단은 절대적인 보안 경계가 아닙니다. 에이전트가 터미널 도구(`bash`)로 `cat ~/.hermes/auth.json` 을 실행하면 여전히 접근할 수 있습니다. 파일 읽기 도구를 명시적으로 존중하는 모델이 자격증명에 손대려 할 때 멈추도록 유도하고, 시도 자체를 로그에 남기는 것이 목적입니다.*

컨트롤 플레인 파일 쓰기 차단

프롬프트 인젝션으로 설정 파일을 덮어쓰는 공격을 막기 위해, `write_file` 도구로 다음 경로에 쓰는 것을 거부합니다.

- `~/.hermes/config.yaml` — 에이전트 설정
- `~/.hermes/auth.json` — 인증 정보
- `~/.hermes/webhook_subscriptions.json` — 웹훅 설정
- `~/.hermes/mcp-tokens/` — MCP 토큰 디렉터리
- `~/.hermes/.env` — 프로바이더 API 키·환경변수

읽기 차단과 마찬가지로, 프로필 모드에서도 활성 프로필 경로와 글로벌 루트 양쪽 모두에 쓰기 차단이 적용됩니다. 글로벌 `.env` 를 프로필 세션이 덮어써서 다른 프로필의 자격증명을 바꾸는 공격 경로를 차단합니다.

이 외에도 `~/.ssh/` , `~/.aws/` , `~/.gnupg/` , `~/.kube/` , `~/.docker/` , `~/.azure/` , `~/.config/gh/` 디렉터리와 `/etc/sudoers` , `/etc/passwd` 등 시스템 파일에 대한 쓰기도 거부됩니다.

커스텀 엔드포인트 API 키 격리

커스텀 프로바이더(`provider: custom`)에 임의의 `base_url` 을 지정해 사용할 때, Hermes는 API 키를 해당 키의 **원래 도메인에만** 전달합니다. 예를 들어 `OPENAI_API_KEY` 는 `api.openai.com` 호스트에만, `OLLAMA_API_KEY` 는 `ollama.com` 호스트에만 전송됩니다. 인식되지 않는 커스텀 호스트에는 키를 전달하지 않습니다.

`pool` 설정이나 프로바이더별 환경변수로 키를 명시적으로 지정한 경우에만 해당 커스텀 엔드포인트에 키가 전달됩니다.

Bitwarden Secrets Manager 연동

API 키를 `~/.hermes/.env` 에 평문으로 보관하는 대신, **Bitwarden Secrets Manager**(BSM)에서 시작 시 가져오는 방식도 지원합니다. 부트스트랩 토큰(`BWS_ACCESS_TOKEN`) 하나로 N개의 프로바이더 키를 대체하며, 키 로테이션이 Bitwarden 웹 앱에서의 단일 변경으로 처리됩니다.

```
hermes secrets bitwarden setup      # 대화형 마법사: bws 설치 → 토큰 입력 → 프로젝트 선택
hermes secrets bitwarden status     # 설정·바이너리·토큰 상태 확인
hermes secrets bitwarden sync       # 수동 가져오기 (--apply로 실제 반영)
hermes secrets bitwarden disable    # 통합 비활성화
hermes secrets bitwarden install    # bws 바이너리만 설치
```

`bws` 바이너리는 첫 사용 시 `~/.hermes/bin/bws`에 자동 다운로드되며, GitHub 릴리스의 SHA-256 체크섬으로 검증됩니다. `sudo`나 패키지 매니저가 필요 없습니다.

```
# config.yaml
secrets:
  bitwarden:
    enabled: true
    project_id: "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx"
    override_existing: true   # BSM 값이 .env 값보다 우선 (기본값)
```

`override_existing: true` (기본값)이면 BSM에서 가져온 키가 `.env`에 이미 있는 같은 이름의 키를 덮어씁니다. 키를 BSM에서 로테이션하면 Hermes를 재시작하는 것만으로 새 키가 적용됩니다. BSM 가져오기에 실패해도(네트워크 불가, 토큰 만료 등) Hermes 시작을 차단하지 않고, stderr에 경고 한 줄만 출력한 뒤 `.env`의 기존 키로 계속 진행합니다.

5계층: 컨텍스트 파일 스캔

AGENTS.md, .cursorrules, SOUL.md, .hermes.md 등 프로젝트 파일을 시스템 프롬프트에 주입하기 전에 프롬프트 인젝션을 탐지합니다.

> 💡 ***마커 변경**: 이전 버전에서 사용자 메시지에 주입되는 안전·중요도 마커는 `[SYSTEM:` 형태였지만, v0.12부터 `[IMPORTANT:`로 바뀌었습니다. 일부 클라우드 콘텐츠 필터(특히 Azure OpenAI)가 `SYSTEM:`을 시스템 프롬프트 위장 시도로 오탐해 차단하는 사례가 있어 이름만 바꾼 것이며, 의미는 동일합니다. 책에서 마커 예시를 따라 쓸 일이 있으면 새 이름을 사용하세요.*

탐지 패턴 (소스 코드 확인)

패턴 ID	탐지 대상
`prompt_injection`	"ignore previous/all/above instructions"
`deception_hide`	"do not tell the user"
`sys_prompt_override`	"system prompt override"
`disregard_rules`	"disregard your/all instructions/rules"
`bypass_restrictions`	"act as if you have no restrictions"
`html_comment_injection`	HTML 코멘트 내 ignore/override/system/secret/hidden
`hidden_div`	`display: none` 스타일의 숨겨진 div
`translate_execute`	"translate into … and execute/run/eval"
`exfil_curl`	curl로 KEY/TOKEN/SECRET/PASSWORD 등 유출 시도
`read_secrets`	`.env` , `credentials` , `.netrc` , `.pgpass` 읽기 시도

비가시 유니코드 탐지

제로 폭 공백(U+200B), 양방향 오버라이드(U+202A-202E), BOM(U+FEFF) 등 비가시 유니코드 문자가 포함된 파일도 차단됩니다.

차단된 파일은 다음과 같이 표시됩니다.

```
[BLOCKED: AGENTS.md contained potential prompt injection (prompt_injection). Content not loaded.]
```

컨텍스트 파일 크기 제한

각 컨텍스트 파일은 최대 20,000자까지만 로드됩니다. 초과 시 앞 70% + 뒤 20%만 유지하고 중간을 잘라냅니다.

11-2. 컨테이너 격리

> ⚠️ ***부분 검증**: v0.16.0 소스 (`tools/environments/docker.py` · `hermes_cli/config.py` · `tools/environments/ssh.py`)를 대조해 보안 플래그·케이퍼빌리티·tmpfs·리소스 제한·설정 키를 확인했습니다. 다만 `--storage-opt` 디스크 제한 지원 여부나 s6-overlay 이미지의 `/run` exec 전환처럼 호스트 환경·이미지 종류에 따라 갈리는 동작은 독자의 환경에서 결과가 다를 수 있습니다.*

- 어떤 백엔드를 고를까
- Docker 백엔드 설정
- Docker 보안 플래그
  - 호스트 사용자로 실행
- 리소스 제한
- 네트워크 격리
- 파일시스템 지속성
  - 지속 모드 (container_persistent: true)
  - 임시 모드 (container_persistent: false)
- 환경변수 전달
- 볼륨 마운트
- SSH 백엔드의 보안 이점
- 터미널 백엔드 보안 비교

운영 환경에서 Hermes Agent를 실행할 때는 컨테이너 백엔드를 사용하는 것이 권장됩니다. 컨테이너 내에서는 위험 명령 승인이 생략되므로, 에이전트가 더 자유롭게 작업할 수 있습니다.

어떤 백엔드를 고를까

이 페이지는 가장 흔히 쓰이는 **Docker** 백엔드를 중심으로 다루고, 그 뒤에 SSH 백엔드, 그리고 마지막에 백엔드별 보안 비교 표를 둡니다. 선택 기준은 다음과 같습니다.

백엔드	적합한 상황
Docker	한 머신 안에서 작업을 격리. 로컬·서버 어느 쪽이든 가장 보편적
SSH	게이트웨이와 명령 실행을 별도 머신으로 분리하고 싶을 때
Modal·Daytona·Vercel Sandbox	클라우드 단발 워크로드(GPU 등). 「설정 파일」 절의 `terminal.backend` 참고
Singularity	HPC 환경에서 root 권한 없이 컨테이너를 띄울 때

이 절은 Docker와 SSH만 자세히 다루며, 다른 백엔드는 설정 키만 같고 운영 원칙은 비슷합니다.

Docker 백엔드 설정

```
# ~/.hermes/config.yaml
terminal:
  backend: docker
  docker_image: "nikolaik/python-nodejs:python3.11-nodejs20"
  docker_forward_env: []   # 빈 배열 = 시크릿 미전달
  container_cpu: 1         # CPU 코어 수
  container_memory: 5120   # MB (기본 5GB)
  container_disk: 51200    # MB (기본 50GB, overlay2+XFS 필요)
  container_persistent: true  # 파일시스템 지속성
```

Docker 보안 플래그

소스 코드(`tools/environments/docker.py`)에서 확인한 실제 보안 플래그입니다.

공식 문서에는 "읽기 전용 루트 파일시스템"이라는 내용이 있지만, 소스 코드에는 `--read-only` 플래그가 없습니다. 파일시스템은 쓰기 가능합니다. 이는 패키지 매니저(pip, npm, apt)가 정상 작동하기 위해 필요합니다.

모든 컨테이너에 적용되는 기본 플래그입니다.

```
_BASE_SECURITY_ARGS = [
    "--cap-drop", "ALL",                    # 모든 Linux 케이퍼빌리티 제거
    "--cap-add", "DAC_OVERRIDE",            # 바인드 마운트 디렉터리 쓰기용
    "--cap-add", "CHOWN",                   # 패키지 매니저의 소유권 변경용
    "--cap-add", "FOWNER",                  # 패키지 매니저의 파일 소유권용
    "--security-opt", "no-new-privileges",  # 권한 상승 차단
    "--pids-limit", "256",                  # 프로세스 수 제한
    "--tmpfs", "/tmp:rw,nosuid,size=512m",          # /tmp: nosuid, 512MB
    "--tmpfs", "/var/tmp:rw,noexec,nosuid,size=256m", # /var/tmp: noexec, 256MB
]
```

`/run`(64MB)은 별도로 분리돼 있습니다. 기본은 `noexec`이지만, s6-overlay 기반 이미지(번들 `hermes-agent:latest` 포함)에서는 init이 `/run/s6/basedir/bin/init`를 실행해야 하므로 `exec`로 마운트됩니다.

여기에 더해, 컨테이너가 root로 시작한 뒤 일반 사용자로 권한을 낮추는 기본 모드에서는 다음 두 권한이 추가됩니다.

```
_PRIVDROP_CAP_ARGS = [
    "--cap-add", "SETUID",   # init이 root → 일반 사용자로 강하할 때 필요
    "--cap-add", "SETGID",
]
```

핵심 포인트를 정리하면 다음과 같습니다.

- **케이퍼빌리티**: `ALL` 제거 후 최소 권한만 복원합니다. 기본 모드에서는 `DAC_OVERRIDE` · `CHOWN` · `FOWNER`에 더해 권한 강하용 `SETUID` · `SETGID`까지 추가됩니다(cap 5개). 뒤에 나오는 `docker_run_as_host_user: true`로 호스트 사용자(`--user`)로 실행하면 권한 강하가 필요 없어 `SETUID` · `SETGID`는 생략됩니다.
- **권한 상승**: `no-new-privileges`로 setuid/setgid 실행 파일을 통한 권한 상승이 불가능합니다. 권한을 낮춘 프로세스가 다시 root로 올라갈 수 없으므로, `SETUID` · `SETGID`를 부여해도 보안 자세는 유지됩니다.
- **PID 제한**: 256개. 포크 폭탄을 방어합니다.
- **tmpfs 크기 제한**: `/tmp`(512MB), `/var/tmp`(256MB), `/run`(64MB)으로 디스크 소진 공격을 방지합니다.
- **/tmp는 exec 허용**: pip, npm 빌드에 필요합니다. `/var/tmp`은 항상 `noexec`이고, `/run`은 기본 `noexec`이지만 s6-overlay 이미지에서는 `exec`로 바뀝니다.

호스트 사용자로 실행

기본적으로 컨테이너는 root로 시작합니다. 지속 모드의 바인드 마운트에서 root가 만든 파일은 호스트에서 root 소유로 남아 다루기 번거로울 수 있습니다. `docker_run_as_host_user`를 켜면 컨테이너를 호스트의 UID:GID(`--user`)로 실행해 이 문제를 피합니다.

```
terminal:
  docker_run_as_host_user: true  # 호스트 사용자로 실행 (기본값: false)
```

이 모드에서는 컨테이너가 처음부터 일반 사용자로 시작하므로 권한 강하가 필요 없어 `SETUID`·`SETGID` 권한이 생략됩니다. POSIX UID가 없는 플랫폼(예: 일부 Windows)에서는 적용되지 않고 경고만 출력합니다.

리소스 제한

```
terminal:
  container_cpu: 2.0     # CPU 코어 수 (float)
  container_memory: 5120 # MB 단위
  container_disk: 51200  # MB 단위
```

> 디스크 제한의 실제 제약
>
> `container_disk`는 Docker의 `--storage-opt size=` 옵션을 사용합니다. 이 옵션은 **overlay2 드라이버 + XFS 파일시스템 + pquota 옵션**이 모두 필요합니다. macOS(Docker Desktop)에서는 아예 건너뛰고, Linux에서도 ext4 기본 설정에서는 작동하지 않습니다. 소스 코드에서 런타임 프로빙으로 지원 여부를 확인한 후, 미지원 시 경고 로그만 출력하고 제한 없이 실행합니다.

네트워크 격리

컨테이너는 기본적으로 네트워크가 켜진 상태로 실행됩니다. `docker.py`에는 `--network=none`으로 네트워크를 끄는 경로가 있지만, 현재 소스에서는 이 값을 직접 바꾸는 전용 설정 키가 연결돼 있지 않습니다. 오프라인 격리가 필요하면 `docker_extra_args`에 플래그를 직접 추가합니다.

```
terminal:
  docker_extra_args:
    - "--network=none"
```

네트워크를 비활성화하면 컨테이너 내에서 외부 접근이 불가능합니다. 패키지 설치가 필요 없는 오프라인 작업에 적합합니다.

파일시스템 지속성

지속 모드 (`container_persistent: true`)

`~/.hermes/sandboxes/docker/<task_id>/` 경로에서 바인드 마운트합니다.

- `/workspace` → `sandboxes/docker/<task_id>/workspace`
- `/root` → `sandboxes/docker/<task_id>/home`

컨테이너를 재시작해도 작업 내용이 유지됩니다.

임시 모드 (`container_persistent: false`)

tmpfs를 사용합니다.

- `/workspace` : rw, exec, 10GB
- `/home` , `/root` : rw, exec, 1GB

컨테이너 종료 시 모든 데이터가 삭제됩니다. 비지속 모드에서는 컨테이너 자체도 `docker rm -f` 로 제거됩니다.

환경변수 전달

```
terminal:
  docker_forward_env:
    - GITHUB_TOKEN
    - NPM_TOKEN
```

`docker_forward_env` 에 명시한 변수만 컨테이너로 전달됩니다. 값은 호스트 환경변수 또는 `~/.hermes/.env` 에서 읽습니다.

docker_forward_env의 보안 주의점

`docker_forward_env` 에 추가한 변수는 컨테이너 내 코드가 읽고 외부로 유출할 수 있습니다. 꼭 필요한 변수만 추가하고, 범위가 넓은 키(예: AWS 루트 키)는 피합니다.

볼륨 마운트

```
terminal:
  docker_volumes:
    - "/host/data:/container/data"
  docker_mount_cwd_to_workspace: true  # 호스트 cwd를 /workspace로 마운트
```

`docker_volumes` 로 추가 바인드 마운트를 설정할 수 있습니다. `docker_mount_cwd_to_workspace: true` 이면 현재 작업 디렉터리가 `/workspace` 에 마운트됩니다. 단, `/workspace` 를 이미 `docker_volumes` 에서 명시적으로 마운트한 경우에

는 건너뜁니다.

SSH 백엔드의 보안 이점

SSH 백엔드를 사용하면, 에이전트가 원격 서버에서만 명령을 실행합니다. 에이전트 자체의 코드나 설정 파일에 접근할 수 없으므로, 자기 수정(self-modification) 위험이 없습니다.

```
terminal:
  backend: ssh
  ssh_host: "sandbox-server"
  ssh_user: "hermes"
  ssh_key: "~/.ssh/hermes_agent_key"
  ssh_port: 22
  ssh_persistent: false  # true면 셸 상태 유지
```

SSH 백엔드의 특징:

- **ControlMaster**: SSH 연결을 재사용하여 이후 명령이 빠릅니다 (ControlPersist=300초)
- **BatchMode**: 대화형 입력 없이 실행됩니다
- **StrictHostKeyChecking=accept-new**: 첫 연결 시 호스트 키를 자동 수락합니다
- **persistent 모드**: `ssh_persistent: true` 이면 하나의 bash 셸을 유지하며 cwd, 환경변수 등 상태가 보존됩니다

SSH에서는 위험 명령 승인이 유지됩니다

Docker와 달리 SSH 백엔드에서는 위험 명령 승인이 생략되지 않습니다. SSH는 원격 머신을 사용하지만, 그 머신이 프로덕션 서버일 수 있기 때문입니다.

터미널 백엔드 보안 비교

백엔드	격리 수준	위험 명령 승인	적합한 용도
local	없음 (호스트 직접 실행)	적용됨	개발, 신뢰할 수 있는 사용자
ssh	원격 머신	적용됨	별도 서버에서 실행
docker	컨테이너	생략됨	프로덕션 게이트웨이
singularity	컨테이너	생략됨	HPC 환경
modal	클라우드 샌드박스	생략됨	확장 가능한 클라우드 격리
daytona	클라우드 샌드박스	생략됨	영구적 클라우드 워크스페이스

11-3. 운영 환경 보안 체크리스트

> ⚠️ ***부분 검증**: 이 페이지는 v0.16.0 소스 코드를 기준으로 재검토했습니다. 보안 플래그·기본값 (redact_secrets·approvals.mode·tirith_fail_open 등)과 컨테이너 격리 동작은 소스에서 직접 확인했으나, 실제 운영 환경 배포 테스트는 거치지 않았습니다.*

- 어떻게 검증했나
- 필수 체크리스트
- 권장 체크리스트
- 웹사이트 차단 설정
- API 키 보안
  - 주요 주의점
- 승인 모드 설정
- 네트워크 격리 구성
- 주기적 감사
- 실전 함정 (gotchas)
  - 1. 디스크 제한이 작동하지 않을 수 있습니다
  - 2. Tirith의 fail-open 기본값
  - 3. SSH 백엔드의 StrictHostKeyChecking
  - 4. 시크릿 마스킹의 한계
  - 5. docker_forward_env 유효성 검사
- 버전별 변경 이력

Hermes Agent를 운영 환경에 배포할 때 확인해야 할 항목들입니다. 공식 문서의 권장사항과 소스 코드 분석에서 발견한 실질적 주의점을 함께 정리합니다.

어떻게 검증했나

체크리스트 각 항목은 다음 기준으로 검토했습니다.

- **공식 문서 매칭**: NousResearch 공식 문서 또는 RELEASE 노트에 같은 권고가 있는지
- **소스 코드 대조**: 보안 플래그·기본값은 `agent/redact.py` · `tools/approval.py` · `tools/website_policy.py` · `hermes_cli/config.py` 소스에서 직접 확인

- **버전 표기**: 항목 안의 버전 메모(v0.9·v0.12 등)는 달라진 지점만 한 줄로 남기고, 자세한 이력은 페이지 맨 아래 「버전별 변경 이력」 절로 분리

체크리스트 본문이 길어지지 않도록, 항목 설명 안에는 방식·이유만 남기고 시점은 별도 절에서 다룹니다.

필수 체크리스트

- [ ] **사용자 인증**: `TELEGRAM_ALLOWED_USERS` 등 플랫폼별 허용 목록을 명시적으로 설정하거나, DM 페어링을 활성화합니다
- [ ] **GATEWAY_ALLOW_ALL_USERS 비활성화**: 운영 환경에서는 절대 `true`로 설정하지 않습니다
- [ ] **컨테이너 백엔드 사용**: `terminal.backend: docker` 또는 SSH/Modal/Daytona 사용 (local 백엔드 지양)
- [ ] **리소스 제한**: `container_cpu`, `container_memory` 설정
- [ ] **API 키 보안**: `~/.hermes/.env` 파일에 `chmod 600` 적용
- [ ] **비root 실행**: root가 아닌 일반 사용자로 게이트웨이 실행
- [ ] **작업 디렉터리 제한**: `terminal.cwd` (config.yaml)로 작업 디렉터리를 한정합니다(과거의 `MESSAGING_CWD` 환경변수는 제거됐고, .env에 남아 있으면 경고가 출력됩니다)
- [ ] **docker_forward_env 최소화**: 꼭 필요한 변수만 컨테이너로 전달합니다
- [ ] **정기 업데이트**: `hermes update`로 보안 패치를 적용합니다

권장 체크리스트

- [ ] **DM 페어링 사용**: 사용자 ID 하드코딩보다 페어링 코드가 유연합니다
- [ ] **command_allowlist 감사**: `config.yaml`의 영구 허용 목록을 주기적으로 점검합니다
- [ ] **로그 모니터링**: `~/.hermes/logs/` 디렉터리의 비인가 접근 시도를 모니터링합니다
- [ ] **웹사이트 차단**: 내부 서비스, 관리자 패널 등에 대한 `website_blocklist` 설정
- [ ] **Tirith 보안 스캔**: 기본 활성화되어 있는지 확인 (`security.tirith_enabled: true`)
- [ ] **시크릿 마스킹**: `security.redact_secrets`는 기본값이 ON입니다(`agent/redact.py`). 운영에서 끄지 않았는지 확인하고, 끈 상태라면 로그·도구 출력의 시크릿 노출 위험을 검토합니다
- [ ] **승인 버튼 사용**: Slack/Telegram 통합을 쓰면 위험 명령 승인이 인라인 버튼으로 뜨므로, 채널을 잠시도 자리를 비워도 잘못된 자동 승인이 발생하지 않도록 채널 권한을 좁게 둡니다
- [ ] **승인 훅 활용**: `pre_approval_request` / `post_approval_response` 훅으로 외부 감사 로그·SOC 시스템에 승인 이벤트를 흘려 보냅니다
- [ ] **Twilio 웹훅 시그니처 검증**: SMS 어댑터를 쓰면 `TWILIO_AUTH_TOKEN`을 설정해 시그니처 검증이 작동하도록 합니다(없으면 webhook 엔드포인트가 누구든 호출 가능)
- [ ] **API 서버 인증**: 내장 API 서버는 노출 시 반드시 토큰 인증을 켭니다(설정 누락에 주의 — 「버전별 변경 이력」 v0.9 항목 참고)
- [ ] **체크포인트 매니저 경로**: `checkpoints/`가 작업 디렉터리 외부 경로 traversal로 빠져나가지 않도록 기본 경로 사용을 권장

웹사이트 차단 설정

에이전트가 접근할 수 있는 웹사이트를 제한합니다.

```
# ~/.hermes/config.yaml
security:
  website_blocklist:
    enabled: true
    domains:
      - "*.internal.company.com"
      - "admin.example.com"
    shared_files:
      - "/etc/hermes/blocked-sites.txt"
```

차단된 URL 요청 시 에이전트에게 에러 메시지가 반환됩니다. `web_search`, `web_extract`, `browser_navigate` 등 모든 URL 관련 도구에 적용됩니다. 공유 파일은 줄 단위로 도메인 규칙을 포함하며, 30초 캐시로 설정 변경이 빠르게 반영됩니다.

API 키 보안

```
# .env 파일 권한 설정
chmod 600 ~/.hermes/.env

# 페어링 데이터 파일도 자동으로 chmod 0600 적용됨
# ~/.hermes/pairing/ 디렉터리 확인
ls -la ~/.hermes/pairing/
```

주요 주의점

- `.env` 파일을 버전 관리에 커밋하지 않습니다
- MCP 서버의 `env` 섹션에는 해당 서버에 필요한 키만 설정합니다
- `docker_forward_env`에는 작업에 필요한 최소한의 변수만 추가합니다

승인 모드 설정

운영 환경에서의 권장 설정입니다.

```
# 컨테이너 사용 시 (위험 명령 승인 자동 생략)
terminal:
  backend: docker

# 로컬/SSH 백엔드 사용 시
approvals:
  mode: smart    # 보조 LLM이 안전한 명령을 자동 승인
```

> **approvals.mode 설정 키 주의**
>
> 설정 키는 `approvals.mode`(중첩 키)이고 기본값은 `"manual"` 입니다(값: `manual | smart | off`). 코드에서는 `"smart"`와 `"off"`만 특별히 처리하고, 그 외 값은 모두 수동 승인으로 작동합니다.

네트워크 격리 구성

최대 보안을 원한다면, 게이트웨이와 명령 실행을 분리합니다.

```
# 게이트웨이 서버 (메시징 연결 담당)
terminal:
  backend: ssh
  ssh_host: "agent-worker.local"
  ssh_user: "hermes"
  ssh_key: "~/.ssh/hermes_agent_key"
```

또는 Docker + 네트워크 비활성화로 완전한 격리를 구성합니다.

```
terminal:
  backend: docker
  docker_image: "nikolaik/python-nodejs:python3.11-nodejs20"
  docker_extra_args: ["--network=none"]  # 외부 네트워크 접근 차단
  container_persistent: false    # 임시 파일시스템 (종료 시 삭제)
  docker_forward_env: []         # 환경변수 미전달
```

주기적 감사

```
# 스킬 목록 확인
ls ~/.hermes/skills/

# 메모리 내용 확인
cat ~/.hermes/memories/MEMORY.md

# 크론 작업 확인
ls ~/.hermes/cron/

# 영구 허용 목록 확인
grep -A 10 command_allowlist ~/.hermes/config.yaml

# 페어링 승인 사용자 확인
hermes pairing list
```

실전 함정 (gotchas)

소스 코드 분석에서 발견한 실질적인 주의점입니다.

1. 디스크 제한이 작동하지 않을 수 있습니다

`container_disk` 설정은 overlay2 + XFS + pquota 조건을 모두 만족해야 합니다. macOS Docker Desktop, 대부분의 Ubuntu 기본 설치에서는 작동하지 않습니다. 소스 코드에서 런타임 프로빙을 수행하고, 미지원 시 경고 로그만 출력하며 제한 없이 실행합니다.

2. Tirith의 fail-open 기본값

Tirith 바이너리가 설치되지 않았거나, 타임아웃이 발생하면 기본적으로 명령을 **허용**합니다. 보안을 강화하려면:

```
security:
  tirith_fail_open: false  # 실패 시 차단 (기본값: true)
```

3. SSH 백엔드의 StrictHostKeyChecking

SSH 백엔드는 `StrictHostKeyChecking=accept-new`로 설정되어 있어, 첫 연결 시 호스트 키를 자동 수락합니다. 보안이 중요한 환경에서는 사전에 `known_hosts`를 설정해 두는 것이 좋습니다.

4. 시크릿 마스킹의 한계

`agent/redact.py`는 알려진 패턴(접두사, 환경변수명, JSON 필드명)만 탐지합니다. 커스텀 API 키 형식이나 비정형 시크릿은 마스킹되지 않을 수 있습니다. 컨테이너 격리와 함께 사용하는 것이 가장 안전합니다.

5. docker_forward_env 유효성 검사

`docker_forward_env`의 항목은 정규식 `^[A-Za-z_][A-Za-z0-9_]*$`로 유효성을 검사합니다. 잘못된 형식의 항목은 경고 로그와 함께 무시됩니다. 하지만 유효한 이름이면 어떤 환경변수든 전달되므로, 목록을 최소화하는 것이 중요합니다.

버전별 변경 이력

체크리스트 안에 흩어져 있던 시점 정보를 한 곳에 모았습니다.

- **redact_secrets 기본값** — 한때 기본 OFF로 운영되던 시기가 있었으나(시크릿 모양의 가짜 일치로 패치 결과가 손상되는 사고 대응), 현재 소스에서는 기본값이 ON입니다(`hermes_cli/config.py`, `agent/redact.py`). 운영에서는 끄지 않은 상태를 유지하는 것이 안전합니다.
- **v0.9** — 내장 API 서버의 미인증 노출 경로가 막힘. 인증 토큰 설정을 누락하지 않도록 운영에서 명시적으로 확인합니다.

11-4. 명령어 허용 목록(command_allowlist) 관리

- allowlist 확인
- allowlist에서 제거
- 에디터 변경

Hermes Agent가 터미널에서 위험하다고 판단하는 명령어를 실행할 때, 사용자에게 승인을 요청합니다. 이때 "Always"를 선택하면 해당 패턴이 `~/.hermes/config.yaml`의 `command_allowlist`에 영구 등록됩니다.

> ⚠️ ***하드라인 패턴은 allowlist로 풀리지 않습니다***: *`HARDLINE_PATTERNS` (복구 불가능한 명령 — 예: 마운트된 디바이스 직접 dd, fdisk·mkfs로 시스템 파티션 포맷 등)는 `command_allowlist` 나 `/yolo` 로도 우회되지 않습니다. 자세한 위계는「보안 모델」절을 참고하세요.*

allowlist 확인

```
# ~/.hermes/config.yaml
command_allowlist:
  - script execution via -e/-c flag
```

`script execution via -e/-c flag`는 Python의 `-c` 옵션(한 줄 코드 직접 실행)을 허용한다는 뜻입니다. 에이전트가 임의의 코드를 동적으로 생성해서 바로 실행할 수 있으므로 위험 패턴으로 분류됩니다.

allowlist에서 제거

```
hermes config edit
```

에디터가 열리면 `command_allowlist:` 아래에서 해당 줄을 삭제하고 저장합니다. 이후 해당 패턴이 나오면 다시 승인 창이 표시됩니다.

에디터 변경

`hermes config edit`는 `$EDITOR` 환경변수를 따릅니다.

```
# 이번 한 번만 vim으로
EDITOR=vim hermes config edit

# 영구 설정 (bash)
echo 'export EDITOR=vim' >> ~/.bashrc
source ~/.bashrc
```

11-5. 멀티 에이전트 구성과 격리

- 왜 에이전트를 나누는가
  - 역할 전문화
  - 장애 격리
  - 보안 경계
- 구성 예시
- 격리 수준 선택
- 에이전트 간 통신
- 실무 체크리스트

하나의 에이전트로 모든 작업을 처리하는 것보다, 역할별로 에이전트를 분리하면 보안, 안정성, 전문성 면에서 유리한 경우가 많습니다. 이 페이지에서는 멀티 에이전트 구성의 동기와 실용적인 구성 방법을 정리합니다.

왜 에이전트를 나누는가

역할 전문화

하나의 에이전트에 코딩, 번역, 일정 관리, 서버 모니터링을 모두 맡기면 메모리와 스킬이 뒤섞입니다. 역할별로 분리하면 각 에이전트가 해당 도메인에 최적화된 메모리와 스킬을 축적할 수 있습니다.

장애 격리

한 에이전트가 크래시하거나 잘못된 루프에 빠져도 다른 에이전트에 영향을 주지 않습니다. 번역 에이전트가 멈춰도 모니터링 에이전트는 계속 작동합니다.

보안 경계

이메일, 캘린더 등 민감한 서비스에 접근하는 에이전트와 외부 웹을 브라우징하는 에이전트를 분리하면, prompt injection으로 한쪽이 오염되더라도 피해 범위를 제한할 수 있습니다.

구성 예시

에이전트	역할	접근 권한
코딩 에이전트	코드 작성, 테스트, PR	로컬 파일 시스템, GitHub
운영 에이전트	서버 모니터링, 로그 정리	SSH, Docker, 알림 채널
리서치 에이전트	웹 검색, 문서 요약	웹 브라우저, 제한된 파일 접근
커뮤니케이션 에이전트	이메일, 캘린더, 메시징	메시징 플랫폼, 캘린더 API

격리 수준 선택

격리 수준은 일반 원칙 — 어디까지 분리할 것이냐 — 으로 정리합니다.

수준	방법	적합한 상황
논리적 분리	에이전트별 별도 workspace와 config 분리	빠른 테스트, 가벼운 용도
컨테이너 격리	Docker로 각 에이전트를 별도 컨테이너. 볼륨과 네트워크 분리	대부분의 개인 사용자 (권장)
완전 시스템 분리	별도 VM 또는 VPS에서 각 에이전트 실행	보안이 중요한 장기 운영

격리 수준이 높을수록 안전하지만 관리 비용도 늘어납니다. 처음에는 논리적 분리로 시작하고, 에이전트의 권한이 커지면 컨테이너 격리로 올리는 것이 실용적입니다.

Hermes Agent에서 구현하는 방법

- 논리적 분리는「프로파일」절의 프로파일 기능으로 에이전트별 workspace를 분리합니다.
- 컨테이너 격리는「컨테이너 격리」절의 Docker 백엔드 설정을 에이전트별로 다르게 줍니다 — 같은 머신에서도 컨테이너·볼륨·네트워크가 분리됩니다.
- 완전 시스템 분리는 SSH 백엔드로 게이트웨이와 명령 실행 머신을 떼어 놓는 방식과 자연스럽게 이어집니다.

에이전트 간 통신

완전히 격리된 에이전트들이 협업하려면 명시적인 통신 채널이 필요합니다. 파일 기반 메시지(공유 디렉터리), HTTP API, 또는 메시징 플랫폼(Telegram, Discord)을 중계로 쓸 수 있습니다. Hermes Agent는 메시징 게이트웨이를 기본 지원하므로 이를 활용하면 자연스럽습니다.

실무 체크리스트

- API 키를 에이전트 간에 공유하지 않습니다
- 각 에이전트에 필요한 최소 권한만 부여합니다 (least privilege)
- 인터넷 접근은 default-deny + allowlist 방식으로 제한합니다
- 각 에이전트의 memory/workspace를 주기적으로 백업합니다

11-6. 프로파일로 환경 분리 운영하기

> ⚠️ ***부분 검증**: v0.16.0 소스 코드(`hermes_cli/profiles.py`, `hermes_cli/main.py`의 profile 서브파서, `cli.py`의 `/profile` 처리)와 공식 문서를 대조해 확인했습니다. CLI 명령·옵션과 경로(`~/.hermes/profiles/<name>`, `active_profile`)는 소스와 일치하며, 멀티 프로파일 실행·게이트웨이 격리를 직접 재현하지는 않았습니다.*

- 프로파일이 격리하는 범위
- 언제 프로파일을 쓰나
- 프로파일 라이프사이클
  - 기존 환경에서 복제하기
  - 일회성으로 다른 프로파일 쓰기
  - 별칭(wrapper) 만들기
  - 프로파일 내보내기·가져오기
- default 프로파일과의 관계
- 활성 프로파일은 어디에 기록되나
- /profile 슬래시 명령

11-5절에서 멀티 에이전트 구성의 동기와 격리 수준을 살펴봤습니다. 이 절에서는 그중 가장 가벼운 분리 수단인 **프로파일** 기능의 사용법을 정리합니다. 프로파일은 한 머신에서 여러 개의 독립된 Hermes 인스턴스를 운영하는 도구입니다.

각 프로파일은 자체 `config.yaml`, `.env`, `SOUL.md`, 메모리, 세션, 스킬, 크론 등을 가집니다. 한쪽 프로파일에서 메모리에 무엇을 적든 다른 프로파일은 영향을 받지 않습니다.

프로파일이 격리하는 범위

프로파일별로 분리되는 데이터와 모든 프로파일이 공유하는 데이터를 정리하면 다음과 같습니다.

구분	항목
프로파일별 격리	`config.yaml`, `.env`, `SOUL.md`, `memories/` (MEMORY.md, USER.md), `sessions/`, `skills/`, `cron/`, `logs/`, `plans/`, `workspace/`, `skins/`, `home/` (서브프로세스용)
모든 프로파일 공유	Hermes 코드(설치된 hermes-agent 패키지), `hermes update` 시 자동 동기화되는 번들 스킬의 새 버전

번들 스킬은 새 버전이 나오면 모든 프로파일에 자동 동기화되지만, 사용자가 직접 수정한 스킬은 덮어쓰지 않습니다.

언제 프로파일을 쓰나

공식 문서에서 안내하는 대표 활용 사례입니다.

- **목적별 에이전트 분리**: 코딩 보조용, 리서치용, 개인 비서용 등 역할별로 메모리·스킬·config가 섞이지 않게 운영합니다(11-5절의 "역할 전문화"와 같은 동기).
- **메시징 게이트웨이별 토큰 분리**: 같은 머신에서 텔레그램·디스코드·슬랙 봇을 각자 다른 계정·토큰으로 동시에 띄울 때, 프로파일별로 `.env`를 두면 토큰 충돌 없이 운영할 수 있습니다(9장 메시징 게이트웨이 참고).
- **기존 환경 백업·포크**: 위험할 수 있는 변경(설정 대수술, 새 모델로 전환 등)을 시도하기 전에 현재 프로파일을 그대로 복제해 두면, 잘못돼도 원본은 그대로 남습니다.

프로파일 라이프사이클

프로파일 관리는 모두 `hermes profile` 서브명령으로 이루어집니다.

```
hermes profile list                 # 모든 프로파일 표시
hermes profile show <name>          # 특정 프로파일 상세
hermes profile create <name>        # 새 프로파일 생성 (빈 상태로)
hermes profile use <name>           # 활성 프로파일 변경 (이후 기본값)
hermes profile delete <name>        # 프로파일 삭제
hermes profile rename <old> <new>   # 이름 변경
```

기존 환경에서 복제하기

비어 있는 새 프로파일이 아니라 현재 환경을 복제해서 시작하려면 `--clone` 계열 옵션을 씁니다.

```
hermes profile create <name> --clone              # 핵심 파일만 복제
hermes profile create <name> --clone-all          # 스킬·세션 등 모든 데이터 복제
hermes profile create <name> --clone-from <other>  # 특정 프로파일을 원본으로
```

`--clone`은 `config.yaml`, `.env`, `SOUL.md`, 메모리 같은 핵심 파일만 복제하고, `--clone-all`은 스킬·세션·로그까지 통째로 복제합니다.

일회성으로 다른 프로파일 쓰기

활성 프로파일을 바꾸지 않고 한 번만 다른 프로파일로 실행하려면 `-p`(또는 `--profile`) 옵션을 사용합니다.

```
hermes -p coding chat               # coding 프로파일로 한 번만 실행
hermes -p research model            # research 프로파일의 모델 설정 보기
```

별칭(wrapper) 만들기

자주 쓰는 프로파일은 별도 실행 파일로 만들어 두면 편합니다.

```
hermes profile alias coding         # ~/.local/bin/coding 생성
coding chat                         # 이후로는 짧게 호출
```

내부적으로는 `hermes -p coding` 을 래핑한 셸 스크립트가 생성됩니다. 제거는 `hermes profile alias coding --remove` .

프로파일 내보내기·가져오기

프로파일을 다른 머신으로 옮기거나 백업하려면 export/import를 씁니다.

```
hermes profile export coding -o coding-backup.tar.gz
hermes profile import coding-backup.tar.gz --name coding
```

default 프로파일과의 관계

`default` 는 별칭이고, 실제 경로는 `~/.hermes` 자체입니다. 즉 **프로파일을 한 번도 만들지 않은 기존 사용자는 자동으로 `default` 프로파일을 쓰는 셈**이고, 별도의 마이그레이션 작업이 필요하지 않습니다.

`hermes profile use default` 를 실행하면 활성 프로파일이 해제되고 다시 `~/.hermes` 로 돌아갑니다.

활성 프로파일은 어디에 기록되나

`hermes profile use <name>` 을 실행하면 `~/.hermes/active_profile` 파일에 프로파일명이 기록됩니다. 다음에 `hermes` 를 실행하면 이 파일을 읽어 자동으로 해당 프로파일을 활성화합니다. 직접 환경 변수로 지정하고 싶다면 `HERMES_HOME=~/.hermes/profiles/<name>` 로 설정하면 됩니다(systemd 유닛이나 Docker 컨테이너에서 유용).

state 격리와 파일시스템 격리의 차이

프로파일은 Hermes 자체의 상태(메모리·스킬·세션·config)만 분리합니다. 파일시스템 접근까지 분리하지는 않습니다 — 모든 프로파일이 같은 사용자 권한으로 같은 파일시스템을 읽고 쓸 수 있고, `terminal.cwd` 같은 작업 디렉터리도 별도로 설정해야 분리됩니다.

따라서 "코딩 에이전트가 개인 비서 에이전트의 파일을 못 읽게 하고 싶다" 같은 강한 격리는 프로파일만으로는 부족합니다. 그 수준이 필요하면 컨테이너 격리(11-2절 참고)나 별도 사용자 계정으로 올라가야 합니다.

`/profile` 슬래시 명령

대화 세션 안에서 `/profile`을 입력하면 **현재 활성 프로파일과 홈 디렉터리를 표시**합니다. 전환 기능은 없습니다 — 프로파일을 바꾸려면 세션을 종료하고 `hermes profile use <name>` 또는 `hermes -p <name>`로 다시 시작해야 합니다.